

Annexure - I

Cover Page

MOVIE RECOMMENDATION SYSTEM

(Content-Based Filtering Recommendation System)

A Project Report

Submitted in partial fulfilment of the requirements for the

Award of the degree of

MASTER OF COMPUTER APPLICATION

By

Shally Ujjwal

322200287



Centre for Distance and Online Education

LOVELY PROFESSIONAL UNIVERSITY

PHAGWARA, PUNJAB

Year

2024

Annexure - II

Declaration by the Student

To whom-so-ever it may concern

I, **Shally Ujjwal, 322200287,** hereby declare that the work done by me on **“Content-Based Filtering Recommendation System for Movies”**, is a record of original work for the partial fulfilment of the requirements for the award of the degree, **Master of Computer Application.**

Name of the Student (Registration Number): Shally Ujjwal , 322200287

Signature of the student:

A handwritten signature in cursive script that reads "Shally".

Dated: 26th- November-2024

Acknowledgement

I am deeply grateful for the guidance, encouragement, and support I received throughout the journey of completing my MCA final year project, *“Movie Recommendation System.”*

First and foremost, I would like to express my sincere gratitude to my project guide, LPU Faculties, for their invaluable insights, constructive feedback, and constant motivation during every stage of the project.

I extend my heartfelt thanks to the Lovely Professional University faculty for providing a conducive learning environment and the technical resources required to complete this project successfully.

Finally, I owe a deep sense of gratitude to my family for their encouragement, patience, and moral support throughout the course of my MCA program, which made it possible for me to achieve this milestone.

This project marks the culmination of my academic journey, and I am truly thankful to everyone who contributed to its successful completion.

Shally Ujjwal

INDEX

S NO	TITLE	P No.
1	Acknowledgement	3
2	Chapter 1: Introduction	5
3	Chapter 2: Review of Literature	6
4	Chapter 3: Implementation of Project	7
5	3.3 Code	8-12
6	Chapter 4: Results and Discussions	13
7	Final Chapter: Conclusion and Future Scope	14

Chapter 1: Introduction

1.1 Overview

Recommendation systems are integral to modern applications, aiding users in navigating vast datasets by providing relevant content suggestions. This project focuses on implementing a content-based movie recommendation system using machine learning and natural language processing techniques.

1.2 Aim and Objectives

- **Aim:** To create a personalized movie recommendation system that delivers relevant suggestions to users based on movie metadata.
- **Objectives:**
 - Design a system leveraging cosine similarity for identifying relationships between movies.
 - Use metadata such as genres, cast, and crew for content-based filtering.

1.3 Importance and Applicability

Recommendation systems are widely used in platforms like Netflix, Amazon, and YouTube, enhancing user engagement by offering personalized suggestions. A movie recommendation system can simplify user decision-making and improve the entertainment experience.

1.4 Scope and Relevance

This project demonstrates the development of a movie recommendation system, which serves as a foundation for more sophisticated approaches like hybrid recommendation systems. The relevance lies in its potential application to streaming platforms, cinema archives, and educational tools.

Chapter 2: Review of Literature

2.1 Previous Studies

- Author: Aggarwal (2016)
Objective: Comprehensive study on content-based filtering techniques.
Findings: Showed the effectiveness of cosine similarity for textual and metadata analysis.

2.2 Research Gaps

While collaborative filtering considers user interactions, it often struggles with new users or content (cold-start problem). Content-based approaches address this but require robust metadata processing, which this project aims to achieve.

Chapter 3: Implementation of Project

3.1 Objectives

- Develop a movie recommendation system leveraging cosine similarity for metadata analysis.
- Deploy the system with a user-friendly interface using Streamlit.

3.2 Methodology

3.2.1 Dataset

The TMDb Movie Dataset was utilized, comprising two files:

- tmdb_5000_movies.csv (movie metadata)
- tmdb_5000_credits.csv (cast and crew details)

3.2.2 Data Preprocessing

- Merged the datasets using movie titles.
- Selected key columns: genres, keywords, cast, crew, and overview.
- Parsed JSON-like string attributes using Python's ast module.
- Generated a unified "tags" column.

3.2.3 Feature Engineering

- Used the Bag of Words (BoW) model via CountVectorizer to create a matrix of movie tags.
- The resulting feature matrix measured 4803 x 5000 (movies x most frequent words).

3.2.4 Similarity Calculation

- Cosine similarity was used to calculate angular distances between movie vectors, determining similarity scores.

3.2.5 Deployment

- Integrated with Streamlit for creating an interactive web interface.
- Fetched movie posters via the TMDb API, enhancing visual appeal.

3.2.6 Tools and Technologies

- Programming Language: Python
- Libraries: Pandas, NumPy, scikit-learn, Streamlit
- External API: TMDb API for poster retrieval

3.3 Code

3.3.1 Model building

```
import numpy as np
import pandas as pd
import os
import ast

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import pickle

# Load data
movies = pd.read_csv('tmdb_5000_movies.csv')
credits = pd.read_csv('tmdb_5000_credits.csv')

# Merge datasets
movies = movies.merge(credits, on='title')
movies = movies[['movie_id', 'title', 'overview', 'genres', 'keywords', 'cast', 'crew']]

# Drop null values
movies.dropna(inplace=True)

# Helper functions
def convert(text):
    L = []
    for i in ast.literal_eval(text):
        L.append(i['name'])
    return L

def convert3(text):
```



```

L = []
counter = 0
for i in ast.literal_eval(text):
    if counter < 3:
        L.append(i['name'])
        counter += 1
return L

def fetch_director(text):
    L = []
    for i in ast.literal_eval(text):
        if i['job'] == 'Director':
            L.append(i['name'])
    return L

def collapse(L):
    L1 = []
    for i in L:
        L1.append(i.replace(" ", ""))
    return L1

# Process data
movies['genres'] = movies['genres'].apply(convert)
movies['keywords'] = movies['keywords'].apply(convert)
movies['cast'] = movies['cast'].apply(convert3)
movies['crew'] = movies['crew'].apply(fetch_director)

movies['cast'] = movies['cast'].apply(collapse)
movies['crew'] = movies['crew'].apply(collapse)

```

```

movies['genres'] = movies['genres'].apply(collapse)
movies['keywords'] = movies['keywords'].apply(collapse)

movies['overview'] = movies['overview'].apply(lambda x: x.split())
movies['tags'] = movies['overview'] + movies['genres'] + movies['keywords'] + movies['cast'] +
movies['crew']

new = movies.drop(columns=['overview', 'genres', 'keywords', 'cast', 'crew'])
new['tags'] = new['tags'].apply(lambda x: " ".join(x))

# Vectorization and similarity computation
cv = CountVectorizer(max_features=5000, stop_words='english')
vector = cv.fit_transform(new['tags']).toarray()
similarity = cosine_similarity(vector)

# Recommendation function
def recomm(movie):
    index = new[new['title'] == movie].index[0]
    distances = sorted(enumerate(similarity[index]), key=lambda x: x[1], reverse=True)
    for a in distances[1:6]:
        print(new.iloc[a[0]].title)

# Example usage
recomm('Gandhi')

# Save model and data
pickle.dump(new, open('movie_list.pkl', 'wb'))
pickle.dump(similarity, open('similarity.pkl', 'wb'))

```

3.3.2 Frontend (streamlit)

```
import pickle

import streamlit as st

import requests


def fetch_poster(movie_id):

    url="https://api.themoviedb.org/3/movie/{}?api_key=8265bd1679663a7ea12ac168da84d2e8&language=en-US".format(movie_id)

    data = requests.get(url)

    data = data.json()

    poster_path = data['poster_path']

    full_path = "https://image.tmdb.org/t/p/w500/" + poster_path

    return full_path


def recommend(movie):

    index = movies[movies['title'] == movie].index[0]

    distances = sorted(list(enumerate(similarity[index])), reverse=True, key=lambda x: x[1])

    recommended_movie_names = []

    recommended_movie_posters = []

    for a in distances[1:6]:

        # fetch the movie poster

        movie_id = movies.iloc[a[0]].movie_id

        recommended_movie_posters.append(fetch_poster(movie_id))

        recommended_movie_names.append(movies.iloc[a[0]].title)


    return recommended_movie_names, recommended_movie_posters


st.header('Movies Recommender System')

movies = pickle.load(open('movie_list.pkl','rb'))
```

```

similarity = pickle.load(open('similarity.pkl','rb'))

movie_list = movies['title'].values
selected_movie = st.selectbox( "Type or select a movie ", movie_list)

if st.button('Show Recommendation'):
    recommended_movie_names, recommended_movie_posters = recommend(selected_movie)
    col1, col2, col3, col4, col5 = st.columns(5)
    with col1:
        st.text(recommended_movie_names[0])
        st.image(recommended_movie_posters[0])
    with col2:
        st.text(recommended_movie_names[1])
        st.image(recommended_movie_posters[1])
    with col3:
        st.text(recommended_movie_names[2])
    st.image(recommended_movie_posters[2])
    with col4:
        st.text(recommended_movie_names[3])
        st.image(recommended_movie_posters[3])
    with col5:
        st.text(recommended_movie_names[4])
        st.image(recommended_movie_posters[4])

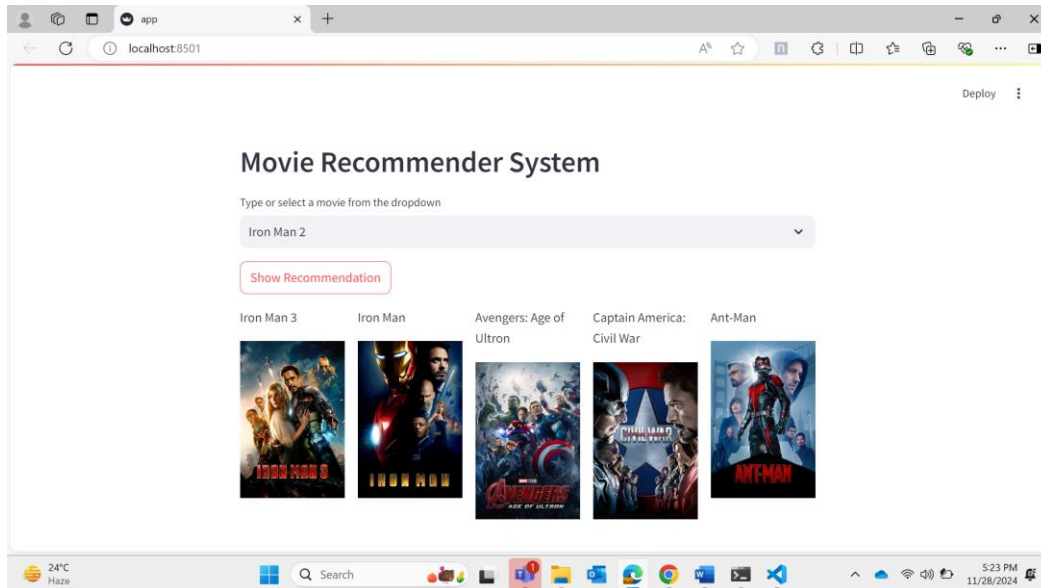
```

Chapter 4: Results and Discussions

4.1 Results

The system successfully provides content-based recommendations. Example:

- Input: "Gandhi"
- Output Recommendations:
 1. Gandhi, My Father
 2. The Wind That Shakes the Barley
 3. A Passage to India
 4. Guiana 1838
 5. Ramanujan



4.2 Discussion

- Accuracy: The recommendations align well with the input movie's theme and metadata.
- User Interface: The integration with Streamlit and TMDb API ensures a seamless and visually appealing experience.
- Challenges: Reliance on content-based filtering limits the system's ability to consider user preferences.

Final Chapter: Conclusion and Future Scope

5.1 Conclusion

The project demonstrates the successful implementation of a content-based movie recommendation system. By leveraging cosine similarity and metadata, the system provides accurate and relevant suggestions. The deployment with Streamlit ensures accessibility and ease of use.

5.2 Future Scope

- **Hybrid Recommendation Systems:** Integrate both content-based and collaborative filtering methods to improve the quality of recommendations.
- **Personalized User Profiles:** Incorporate user-specific data, such as viewing history and ratings, to improve personalization.
- **Advanced Techniques:** Explore deep learning-based models like recurrent or convolutional neural networks for better feature extraction and similarity computation.